

USAIL :: AI4S PDE Neural-Operator Agent

Methodology: ReAct-Driven Auto-Research over PDE Tasks

Team USAIL -- 4th World Scientific Intelligence Challenge, Track 4 (AI4S Agent CNS)

1. Overview

We treat the AI4S PDE neural-operator challenge as an autonomous research problem rather than a static modeling task. Our agent is a ReAct-style loop wrapped around an LLM (Claude Opus 4.7 by default; the architecture is provider-agnostic), with three coupled stages -- Plan, Act, Observe -- repeated until a quantitatively verified solution exists for each of the three tasks (Task 1: fixed-Nu 1D Burgers, Task 2: multi-Nu 1D Burgers from scratch, Task 3: Kuramoto-Sivashinsky from short observations).

All scientific decisions -- model architecture, fine-tune strategy, validation protocol, hyperparameter search budget -- are produced by the agent. Human input is restricted to providing the task description, the data, and the sandbox in which code executes. No model weights are hand-tuned and no per-task code is hand-edited.

2. The ReAct Loop

Each task is solved by an episode of the following loop, with a search budget of N solve-steps (default N=3; configurable). At every step the agent produces a Plan, executes an Action, then ingests an Observation, and the loop chooses what to do next.

Plan

Given the task description, the cumulative experiment log, and the history of previous attempts, the LLM proposes (a) a one-paragraph natural-language plan describing the next intervention, and (b) a complete, self-contained Python script that implements it. The plan is constrained to a single atomic improvement -- one architectural change, one loss term, or one hyperparameter sweep -- so that each iteration produces an interpretable A/B comparison.

Act

The proposed script is executed verbatim inside a hermetic sandbox (directory: `<run-root>/workspace/<task>/sandbox/`). The sandbox has symlinks into the task data, the official checkpoints (Task 1 only), and the pre-downloaded training corpus where applicable. All stdout, stderr, raised exceptions, and produced artifacts (HDF5 predictions, inference-time text files) are captured. A separate logging proxy transparently intercepts every LLM call the agent makes and writes it to a JSONL log -- this log is the official evidence trail.

Observe

A judge LLM call (with `response_format=json_object`) reviews the executed script, its stdout, the produced prediction file, and the computed validation metric. It returns a structured `ReviewResult` (`is_buggy`, `summary`, `metric_value`, `lower_is_better`). The agent's search policy then decides: continue to improve the best successful node, or debug the most recent failed node, or draft a fresh solution. State persists across steps so unsuccessful directions are remembered and not re-tried.

Concretely, this is the AIDE workflow (Anthropic Industrial Development Engineer) wired with Claude Opus 4.7 as the underlying LLM via an OpenAI-compatible proxy. We patched the proxy's JSON validator to accept Claude's habit of wrapping JSON responses in markdown fences, which was a recurring source of failed judge calls before this fix.

3. Scientific Decision-Making

Within the ReAct loop, the agent makes the four canonical research decisions autonomously for each task:

- Problem Structuring. The agent reads the task description, summarizes the PDE, the input/output

contract, the evaluation metric (segment-weighted MSE plus, for Task 3 segment 3, a Lorentzian / Frechet distributional score), the hard constraints (no pretrained checkpoints for Tasks 2 and 3; no validation data in any backward pass; 2-minute inference cap for Tasks 2 and 3), and the time budget.

- **Architecture Selection.** For all three tasks, the agent chooses a PDEBench-aligned Fourier Neural Operator (FNO1d) implementation inlined directly into the script (so there is no dependency on the PyPI neuraloperator package, whose v2 state-dict layout is incompatible with the released Task 1 checkpoint). For Task 1 it warm-starts from the released Nu=0.001 checkpoint; for Tasks 2 and 3 it trains randomly initialized weights from scratch.
- **Hypothesis Iteration.** After each Observe step, the agent forms a specific, falsifiable hypothesis ("the long-horizon segment fails because the 1-step training loss does not regularise autoregressive stability; a 5-step rollout loss should fix it") and the next Plan implements an experiment that confirms or denies it. Failed hypotheses are recorded; the search policy does not re-propose them.
- **Stop Criterion.** An episode ends when (a) the validation metric is below a task-specific target, (b) the search budget is exhausted, or (c) the wall-clock budget runs out. The best successful node's prediction file is selected for submission; ties are broken by the long-horizon segment's score, since that carries 50% of the official weight in all three tasks.

4. Task-Specific Findings

Task 1 (fixed Nu=0.001 Burgers)

Zero-shot rollout of the released FNO checkpoint scores `weighted_score = 0.4457` on val. The agent verified that the checkpoint was trained with 1-step loss only, which causes exponential divergence past frame ~50 of the 190-step rollout. The agent's fine-tune intervention (5-step rollout loss, AdamW $\text{lr}=1\text{e-}4$, gradient clip 1.0, 2 epochs over the 10000-sample PDEBench training file downloaded into data/) reduces the val `weighted_score` to 0.000340 -- a 1300x improvement -- in approximately 70 seconds on CPU. All five hyperparameter choices (data-slicing pattern `u[:, :200:5, ::4]`, 5-step horizon, gradient clipping, random temporal window start, AdamW $\text{lr}=1\text{e-}4$ $\text{wd}=1\text{e-}5$) were verified to be individually load-bearing.

Task 2 (multi-Nu Burgers, from scratch)

Training data spans Nu in $[1\text{e-}4, 1\text{e-}2]$ across 3000 trajectories. The agent chooses a nu-agnostic FNO baseline (rather than conditioning the model on nu), reasoning that (a) the 10-frame input window already encodes the dynamical regime, and (b) a noisy nu estimator at test time can hurt more than no nu information. Training uses a horizon curriculum 1 -> 5 -> 10 over 30 cosine-annealed epochs with input mean/std normalization computed on the training split only.

Task 3 (Kuramoto-Sivashinsky)

KS is fundamentally chaotic: the Lyapunov time on this setup is approximately 10-20 model steps, and exact pointwise prediction past that horizon is physically impossible. The agent identifies that the seg3 score (steps 200-399, weight 50%) is judged by `max(Lorentzian, Frechet)`, which is a distributional metric -- so the optimization target shifts from pointwise accuracy to spectral fidelity. The agent uses a longer horizon curriculum (1 -> 5 -> 10 -> 20), wider Fourier modes (`modes=24`, because the `u_xxxx` term lives at high wavenumbers), and a spectral MSE side term in the loss. The input window is 20 frames (vs. 10 for Burgers) because the chaotic dynamics need more context.

5. Reproducibility & Audit Trail

Every artifact required for an audit is produced automatically:

- `task{N}_pred.hdf5` -- the (N, T, 256) prediction tensor; the first observed frames are copied verbatim from the test input to satisfy the $1\text{e-}3$ tolerance check.
- `task{N}_inference_time.txt` -- a plain text file containing the test-set rollout wall-clock in seconds, written by the agent immediately after the `time.perf_counter()`-wrapped rollout call. Also printed to stdout as `INFERENCE_TIME=<sec>` for redundancy.
- `task{N}_time.csv` -- assembled post-run by the submission packager from (a) the inference-time text

file above and (b) the LLM call log timestamps. `train_time` is the wall-clock delta between the first and last LLM call in the session, which includes all agent thinking time as required by the official rules.

- `task{N}_logs.log` -- a JSONL file where each line is one complete LLM call (timestamp ISO-8601-with-timezone, `elapsed_seconds`, `request`, `response` or `tool_calls`). Produced by the official proxy at `task_log_sample/openai-log/proxy.py`. Together with the `code/` directory this is the auditable evidence that the submitted code was agent-generated.
- `code/` -- the final, agent-generated Python script(s) that produced the predictions. Every line in `code/` can be traced to a specific LLM response in `task{N}_logs.log`.

6. Limitations & What We Did Not Do

- We did not implement an outer AutoML search loop. The agent explores hyperparameters within the ReAct iteration budget but does not run a separate Optuna / Bayesian sweep; this is left to future work and would mostly affect Task 3 where the chaotic regime has more sensitive hyperparameters.
- Seed ensembling is implemented as a description-level recommendation but not always exercised; an N-seed average typically yields a few percentage points on the weighted score at near-zero cost.
- The `proxy.py` logger only parses OpenAI-compatible responses. For native Anthropic `/v1/messages` traffic, the parser needs extension; we sidestep this by routing Claude calls through an OpenAI-compatible shim.

7. Submission Layout

```
submission/
??? submission.json           # {"submission_id": "usail", ...}
??? methodology.pdf          # this document
??? task1_pred.hdf5          # Task 1 predictions
??? task1_time.csv           # Task 1 train+inference time
??? task1_logs.log           # Task 1 LLM call log (JSONL)
??? task2_pred.hdf5          # Task 2 predictions
??? task2_time.csv
??? task2_logs.log
??? task3_pred.hdf5          # Task 3 predictions (optional)
??? task3_time.csv
??? task3_logs.log
??? code/                    # agent-generated source
??? ...
```